

Сверточные нейронные сети

Высшая Школа Цифровой Культуры
Университет ИТМО
dc@itmo.ru

Содержание

1	Свертки в нейронных сетях	2
1.1	Введение	2
1.2	Немного об операции свертки	3
1.3	Сверточные слои в нейронной сети	5
1.4	Padding, stride	6
1.5	Субдискретизация i.e. pooling	7
1.6	А что на выходе?	7

1 Свертки в нейронных сетях

1.1 Введение

Здравствуйтесь, уважаемые слушатели! В предыдущих лекциях мы обсуждали архитектуры многослойных полносвязных (fully-connected) нейронных сетей, основанных на математическом приближении нейрона – перцептроне. Кроме того, мы узнали, как реализовывать эти архитектуры на практике в Python, а также поговорили о важных вопросах обучения построенных моделей. Казалось бы, неужели это все? Конечно же нет. Оказывается, искусственные нейронные сети можно наделить еще большим «интеллектом».

Сверточные нейронные сети, о которых мы будем говорить в этой лекции, оказываются совершенно новой с точки зрения идея и подхода математической моделью. Они позволяют учитывать собственную **внутреннюю структуру**, присущую данным. Чтобы понять, что кроется за этими словами, давайте вспомним то, что мы делали, строя несколько лекций назад самую первую нейронную сеть для решения задачи классификации рукописных цифр. Мы «распрямляли» наше изображение, подавая на вход сети вектор некоторой длины, формируя, тем самым, входной слой. Далее в каждом слое повторялось всегда одно и то же: некоторое линейное преобразование + сдвиг (иначе – аффинное преобразование), а затем применение нелинейной функции активации.

Эта кажущаяся простота и интерпретируемость, однако, является в том числе и достаточно очевидной проблемой: при изложенном подходе мы (точнее – наши аффинные преобразования) совершенно не учитываем структуру входного изображения или, как принято говорить, топологию данных. А ведь эта топология не просто присутствует во многих данных, но и играет ключевую роль: вы же понимаете, что взаимное расположение пикселей на картинках очевидным образом зависит от того, какая цифра на картинке изображена, а значит эта информация оказывается чрезвычайно важной при обучении «достойной» модели.

Еще раз вспомним, как мы поступали раньше. Раньше мы бросали данные в нашу сеть, а сама сеть выступала в роли «слепого котенка»: она должна была не только сформировать какие-то хорошие фичи при обучении, но и понять что-то о форме цифры, догадаться в принципе о наличии такой штуки, как формы, о существовании находящихся рядом (на картинке) пикселей и их возможной корреляции, имея при этом на входе лишь вектор интенсивностей пикселей. Непростая задача, не так ли?

Кроме того, ранее используемый подход оказывается совершенно непригодным к стандартным для изображений (да и для многих других типов данных) ситуациям: сдвиг, поворот одного и того же объекта. Нам-то

с вами понятно, что чтобы определить, что на фотографии запечатлена собака, совершенно не важно, как далеко ее правое ухо отстоит от верхнего края фотографии: на 200 или на 300 пикселей. Более того, мы бы узнали собаку и на совсем обрезанной фотографии. По чему? Да по локальным признакам, по цельной информации, полученной от некоторого связного участка картины. Но то, что понятно нам, совершенно чуждо ранее построенной архитектуре. Так мы и приходим к ключевой идее сверточных нейронных сетей – локальной обработке участков входного набора данных некоторым количеством более экономных нейронных сетей, обучаемых замечать связные детали. Технические аспекты этой идеи хорошо видно на анимации 1. Говоря общо, по матрице, отвечающей входному изображению размера 5×5 , проходятся ядром (фильтром, матрицей весов) размера 3×3 , получая новую матрицу (карту признаков) размера 3×3 . Описанная операция, которая из одной, входной матрицы, создает каким-то определенным образом вторую, называется сверткой; отсюда и название – сверточные нейронные сети. О математических основах этой ключевой для архитектуры операции мы сейчас и поговорим.

Рис. 1: Сверточный слой

1.2 Немного об операции свертки

Итак, давайте неформально поговорим о том, что такое операция свертки. Сначала будем считать, что входные данные X представляют собой двумерную матрицу (это соответствует, например, матричному представлению черно-белого изображения). Пусть, к тому же, дана матрица весов W , «помещающаяся в исходную». Тогда операция свертки этих двух матриц может быть осуществлена согласно схеме, изображенной на рисунке 2. По сути, мы ходим «скользящим окном», или трафаретом, по матрице, и находим скалярное произведение (фробениусово) накладывающихся друг на друга подматриц. Например, сверткой двух матриц

Рис. 2: Двумерная свертка

$$X = \begin{pmatrix} 1 & 2 & 3 & 1 \\ 1 & 4 & 0 & 0 \\ 2 & 2 & 0 & 2 \\ 0 & 4 & 1 & 3 \end{pmatrix} \quad \text{и} \quad W = \begin{pmatrix} 1 & 2 & 0 \\ 0 & 3 & 0 \end{pmatrix}$$

будет матрица

$$X * W = \begin{pmatrix} 17 & 8 \\ 15 & 4 \\ 18 & 5 \end{pmatrix}.$$

Проверьте, что вы поняли, как осуществляется свертка.

Итак, согласно описанному алгоритму, в результате свертки матриц $X_{[m \times n]}$ и $W_{[r \times k]}$, $r \leq m$, $k \leq n$, получается матрица $(X * W)_{(m-r+1) \times (n-k+1)}$. Можно посмотреть и на пошаговую анимацию описанного алгоритма (анимация 3).

Рис. 3: Двумерная свертка

Конечно, часто мы будем иметь дело со входными данными X , которые представляют собой не двумерную матрицу, а матрицу большей размерности. Такие объекты в теории нейронных сетей называются тензорами. Тензор можно воспринимать как матрицу, у которой элементы – векторы. Например, цветное изображение 4×4 в формате RGB имеет тензорное представление, наглядно представленное на рисунке 4. Каждому пикселу, то есть каждому элементу двумерной матрицы размера 4×4 , соответствует вектор из трех чисел – интенсивностей по каналам R, G и B, соответственно. Понятно, что размер матрицы соответствует размерам изображения, а на рисунке приведен лишь весьма частный пример, для наглядности.

Рис. 4: Тензорное представление RGB изображения

Предположим теперь, что входные данные X представляют собой тензор размера $[m \times n \times l]$. Тогда свертка с (теперь уже) тензором весов размера $[r \times k \times l]$, $r \leq m$, $k \leq n$ – это тензор $X * W$ размера $[(m-r+1) \times (n-k+1) \times 1]$. Итого, результат свертки – это «плоская» карта. Сама свертка производится также, как и ранее: мы «ходим» тензором W по тензору X , осуществляем скалярное (фробениусово) произведение накладывающихся друг на друга тензоров, а результат записываем в соответствующую ячейку нового, «плоского» тензора. Наглядно вся процедура свертки показана в анимациях 5 и 6. На них видно, что тензор весов разделен на три «плоских» тензора (это эквивалентно описанному выше алгоритму), каждый из которых производит свертку с каждым соответствующим слоем входного тензора, а затем результаты складываются; в результате всех этих манипуляций и получается «плоская» карта. Понятно, что все описанные соображения обобщаются на тензоры произвольных размеров. Мы не будем приводить формулы в общем виде, и вместо этого, наконец-то, изучим, как операция свертки позволяет строить сверточные слои в нейронных сетях.

Рис. 5: Трехмерная свертка

Рис. 6: Трехмерная свертка

1.3 Сверточные слои в нейронной сети

Итак, а что же с нейросетью? Наверняка вы уже поняли, как создается сверточный слой, рисунок 7. Имея входные данные, задается тензор весов (фильтр), последняя размерность которого совпадает с последней размерностью тензора, отвечающего входным данным. Далее вычисляется свертка, и на выходе получается некоторая карта. В конце, ко всем элементам карты добавляется одно и то же число – сдвиг (аналогично добавлению $-w_0$ в случае перцептрона).

Рис. 7: Сверточный слой

Замечание 1.3.1 При обучении нейронной сети неизвестными (настраиваемыми параметрами) являются компоненты фильтра и константа, отвечающая за сдвиг. Давайте вычислим количество параметров описанной сети. Понятно, что их $3 \cdot 3 \cdot 3 + 1 = 28$, при этом на выходе мы получаем 36 значений. Что бы было в случае полносвязной нейронной сети? На вход бы подавался вектор из $8 \cdot 8 \cdot 3 = 192$ чисел. Чтобы получить 36 выходов, нам бы потребовалось использовать 36 нейронов. Каждый нейрон имеет 193 веса (не забываем про сдвиг!). Итого, в случае полносвязной нейронной сети нам бы пришлось искать $36 \cdot 193 = 6948$ параметров. Посмотрите на отличие в количестве настраиваемых параметров даже для такого сравнительного небольшого входа!

Теперь, конечно, обобщения. Понятно, что в слое может быть (и обычно бывает) несколько фильтров, в итоге на выходе получается не одна карта, а множество, рисунок 8. Число фильтров – это, естественно, гиперпараметр, настраиваемый исследователем. Как вы, надеюсь, поняли из введения, задача фильтра – выучить некоторый паттерн, и дальше выдавать наибольшие значения при обнаружении его на изображении, рисунок 9. Тем самым, количество фильтров в слое отвечает, грубо говоря, количеству паттернов, которые вы хотите найти во входных данных.

Рис. 8: Сверточный слой с несколькими фильтрами

Рис. 9: Обнаружение паттерна фильтром

Теперь давайте поговорим, а как создавать много слоев? Так как все описанные операции оказываются линейными, то нет смысла плодить несколько сверточных слоев подряд, не добавляя нелинейность между ними. Нелинейность, как и раньше, обеспечивается, например, функциями активации, которые применяются к картам покомпонентно. Иными словами, тензор с нелинейностями имеет ту же размерность, что и карты, получившиеся после сверточного слоя. Так как результатом покомпонентного применения нелинейности является снова некоторый тензор (или еще активационная карта), то он и является входными данными для следующего, быть может сверточного слоя, который формируется совершенно аналогичным образом, но, конечно же, со своими фильтрами. И так далее. Таким образом, мы пытаемся достигнуть некоторой иерархичности. Скажем, фильтры первого слоя пытаются обнаружить какие-то черточки, направленные под разными углами, фильтры второго слоя пытаются «собирать» черточки вместе, and so on. Итого, чем глубже слой, тем больше комбинаций изначально выделенных фичей наша сеть пытается разучить для дальнейшего, надеемся, успешного применения.

1.4 Padding, stride

Отвлечемся еще на некоторые технические, но важные детали. При выполнении операции свертки могут варьироваться параметры, называемые padding (дополнение) и stride (шаг).

Параметр padding отвечает за равноправие между пикселями. Согласно идеям, изложенным ранее, мы ходим фильтром по нашим данным по причине того, что мы хотим учесть и обработать каждый пиксель не сам по себе, а вместе с некоторой его окрестностью, то есть чтобы учесть топологию данных. В то же время, если мы запрещаем фильтру выходить за пределы тензора, то как минимум элементы тензора, находящиеся на границе, никогда не будут «центром» соответствующей окрестности (кстати, из-за этого уменьшается и размер карты по сравнению с размером входных данных). Чтобы исправить это недоразумение, мы можем дополнить исходный тензор достаточным количеством нулей (или каких-то других одинаковых значений) – установить значение padding – и, тем самым, позволить каждому пикселю быть «центром» окрестности. Наглядно это можно увидеть на анимации 10. Впрочем, бывают разные версии описанного трюка. Например, иногда добавляют такое количество нулей, чтобы каждый элемент входного тензора «побывал» в каждом элементе фильтра. Все это настраивается внутри инструментов. Подробнее с вариантами можно ознакомиться в статье [Convolution arithmetic tutorial](#).

Параметр strides, в свою очередь, задает шаг фильтра по изображению.

Рис. 10: $\text{Padding} = 1$

Понятно, что он, по своей сути, показывает, насколько часто мы хотим применять наш фильтр: к каждому ли пикселю, или часть пикселей можно пропускать. Наглядно все видно на анимации 11.

Рис. 11: $\text{Stride} = 2$

Конечно, обе эти идеи можно комбинировать, анимация 12.

Рис. 12: $\text{Padding} = 1, \text{Stride} = 2$

1.5 Субдискретизация i.e. pooling

Итак, мы с вами научились выполнять разные типы свертки. После свертки, как было сказано ранее, можно применять ту или иную функцию активации (поэлементно к элементам карт). Однако, и это еще не все. Обычно в сверточном слое выделяют еще одну, третью операцию, так называемую субдискретизацию (pooling). Идейно эта операция вводится вот почему: зачастую нам интересен факт наличия в данных того или иного признака, а не точное его расположение (не точные координаты). Например, при распознавании той же собаки на изображении, нам намного важнее понять, есть ли на изображении собака, и если есть, то какой породы, нежели точно определить координаты начала правого уха. Поэтому имеет смысл «обобщить» выделенные признаки, потеряв немного в информации о точном расположении, но заметно сократив размерность данных. Кроме того, такая операция позволяет создать устойчивость к небольшим трансформациям, например, к сдвигам.

Одной из самых частых операций является так называемая операция max-pool. Она проходится с некоторым шагом stride по каждой локальной группе объектов активационной карты и выбирает из них максимум. Вместо кропотливого объяснения, идею проще всего понять из рисунка 13. Итого, сверточный слой состоит из, вообще говоря, трех операций: свертка, нелинейность, субдискретизация.

Рис. 13: max-pool 2×2 , $\text{stride} = 2$

1.6 А что на выходе?

Итак, мы научились строить архитектуру сверточных нейронных сетей. Но что на выходе? Обычно, в конце, после последней операции pooling, полу-

чившийся тензор распрямляется в вектор и создается несколько полносвязных слоев, в результате чего получается, например, решение задачи классификации, рисунок 14. Все операции здесь дифференцируемы, так что для вычисления градиентов вполне подходит уже описанный ранее `backprop`. Формулы, однако, мы выписывать не будем.

Рис. 14: Классификация